Mapping Capabilities with LLMs: Requirements and Feasibility Assessment of Capability Broker

Adrian Olbricht[1[0009-0006-5009-6591]], Jennifer Skopac[1[0009-0007-5846-4640]], Jan Lundberg[1,2 [0009-0008-0485-6005]], Jelena Zdravkovic[1[0000-0002-0870-0330]] and Janis Stirna[1[0000-0002-3669-832X]],

[1] Stockholm University, Kista, Sweden
{adrian.olbricht, jennifer.skopac, jan.lundberg, jelenaz, js,}@dsv.su.se
[2] Swedish Defence University, Stockholm, Sweden

Abstract. Capabilities provide a structured and stable business-centric view of what an organization is able to do. Capability has become a standard design element of enterprise architecture frameworks such as NAF, TOGAF, ArchiMate and many others. Despite its usefulness, clear modularity, and purpose of the capability notion, many companies struggle becoming capability-aware and being able to continuously and efficiently manage their capability portfolios. To support this process, this paper presents requirements and an early design prototype of Capability Broker – an LLM based tool for discovering capabilities from organizational documents and creating capability models. The feasibility of the tool is demonstrated with an example case.

Keywords: LLM, Enterprise Modeling, Capability, NAF, AI

1 Introduction

Strong business performance relies on a well-designed, integrated enterprise architecture (EA) to control organizational complexity. EA aims to provide a holistic view of an enterprise through aligning multiple parts of the organization, such as different actors, products and services, organizational structures, and business processes [1, p. 3]. The concepts of EA have been widely adopted over the last few decades to manage complex organizational designs. However, the way EA connects business and IT-related aspects is often perceived as inflexible [**Error! Reference source not found.**, p. 7]. As organizations navigate dynamic business landscapes, adaptive ways of working have become fundamental in development and management. The need for flexibility in managing both internal and external context changes has therefore sparked an interest in capability-oriented thinking in organizations [**Error! Reference source not found.**, p. 5].

Capability is generally recognized as a concept in business architecture. Capabilities are often referred to as the organizational building blocks, i.e. what is the organization's ability and capacity to achieve objectives in a certain context, see for example [2 p. 5;

3, 4] for capability definitions. In the context of military-oriented Capability-Based Planning Andersson et al. [4] liken the use of capabilities to building with Lego, as each capability is a piece that helps you build new solutions when needed. Furthermore, business capabilities focus on what an organization is able to do. They provide a common language and understanding of which areas of the business should be emphasized [5]. This common language and understanding of capabilities are integral parts of the strive for continuous results in the context of changing business environments [6].

Although there are strong reasons for modern organizations to adopt capabilities, the time investment required to identify, develop, and manage them may discourage them from fully engaging in capability management. A common starting point towards capability management is a capability map. However, creating them for complex organizations is a tedious task, because it requires analyzing existing business design, defining and modeling each capability, relating it with similar capabilities, and finally placing it into the capability map. As a consequence, many companies do not manage to create the map, or if they do, the map often becomes static and out of date as the organization evolves. Moreover, identifying new capabilities from emerging business documents and linking that to the existing capability map needs to be supported.

To this end [6] proposed Capability Broker – a tool for capability discovery and mapping with the help of the Large Language Model (LLM). The motivation for the proposal was the potential and ability of LLMs to analyze large document corpus and support conceptual modelling [7which are the types of tasks that need to be done as part of capability mapping.

The project follows the design science DSR research framework [**Error! Reference source not found.**] and considers Capability Broker as its design artifact. According to [8] DSR is iterative and undergoes five phases - problem explication, requirements definition, design and development, demonstration, and evaluation. Currently Capability Broker is at the stage when the definition of the initial set of requirements has been completed and the initial feasibility of the prototype has been carried out. Therefore, the goal of the paper is *to present the initial cycle of the development of Capability Broker and more specifically, requirements and the initial feasibility assessment of capability discovery and mapping with LLMs in the military domain*.

The motivation for the choice of this domain is the need to have (i) sufficiently complex organization's activities, (ii) the need to work with capabilities which is common the military domain, (iii) availability of reasonably elaborated descriptions of organization's activities on various levels of details such as NATO's Allied Join Publications (AJP) and rich pictures describing operations on various levels of detail, and (iv) the need use of a defined way of representing capabilities such as the NAF to be used for military organizations of the NATO member states.

The rest of the paper is structured as follows. Section 2 gives a brief background to the capability mapping and the initial vision of Capability Broker. Section 3 presents requirements for Capability Broker. Section 4 presents and discusses example prompts used in the project. Discussion and concluding remarks are presented in section 5.

2 Background

2.1 Capability Modeling and Mapping

Capability is the ability and capacity that enable an enterprise to achieve a business goal in a certain context [10]. Capabilities are also at the core of Capability Based Management [11]. There are two general approaches to modeling capabilities, namely, as capability maps and as part of an enterprise model as it is done in modeling approaches such as CDD [2]. Capability maps are hierarchical i.e. they include several levels of capabilities (e.g. level 1, level 2, level 3…), where each level is a refinement of the previous one. At the top level, capabilities are typically highly stable. Lower-level capabilities are more operational and related with the IT applications supporting them. Each capability presented in the map has its internal design – it has specified how the capability will be delivered by processes, resources, and services to meet a goal and in which situation (context) is capability suitable to apply.

NAF [3] covers both approaches by offering some viewpoints that are mostly used for capability mapping (Capability Taxonomy C1, Enterprise Vision C2, and Capability Dependencies C3) and other viewpoints for linking capabilities with means of measurement (Performance Parameters C7), and realization (Standard Processes C4, Node Types L1, Resource Types P1, Resource Structure P2, Resource Connectivity P3 to name a few).

2.2 Overview of the Capability Broker

The vision for the Capability Broker was proposed in [6]. Its architecture consists of these conceptual components – capability map, LLM component for capability discovery, searching, and discovery of capabilities, and capability recommender. The tool should also have a knowledge repository for storing documents and models created.

The process of the capability broker's use for capability management is envisioned as Figure 2 depicts it on a high-level:

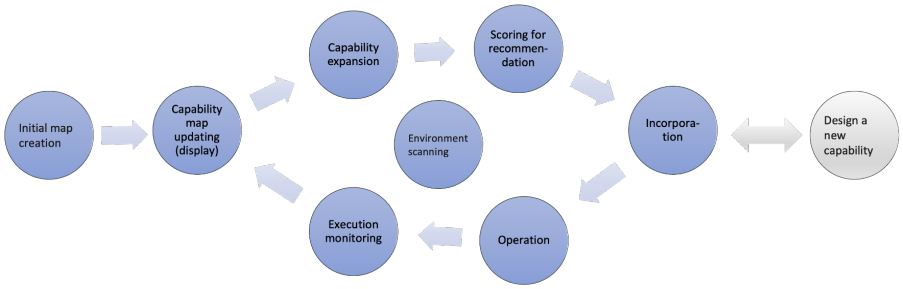


Fig. 1. Overview of the work process supported by the Capability Broker

The use of the process steps depends on which state of advancement of the capability use the organization finds itself.

3 Requirements for the Capability Broker

The current project is structured in iterations and hence we first elaborate requirements for the steps needed for the initial capability map and discovery of potentially related capabilities in source documents. The more complete set of requirements and other supplementary material such as models generated is available on GitHub [12].

In [6] we propose these three steps (see also fig.1):

Initial map creation: This task requires entering the organization's capability map into the capability broker. This can be done in two ways. (i) If the organization (or some its business unit) does not have an explicit capability map, this task involves building an initial Capability Map in the broker. The organization can also use the LLM component and enter its business descriptions as prompts to get suggestions for possible capabilities at the top level of capabilities. Subsequent prompting can then be used to refine the level 1 capabilities to sub-capabilities at level 2 and below. (ii) If the organization has a capability map, then it needs to be entered into the broker i.e. for obtaining the machine-readable format, also unified from the perspective of use.

Capability Map display: this task provides the view of the existing capabilities to the expert for looking for a needed capability browsing from higher to lower levels.

Capability expansion: the task enables identification of new capabilities when the capability is not found in the map. E.g., this can be by analyzing descriptions of new technologies or business offerings from different organizational units.

Requirements for Initial map creation.

The following requirements categories support this phase - (1) input text such as prompts and instructions, (2) file uploads for the purpose of creating a knowledge repository that focuses the LLMs response, and (3) capability map creation.

Input to the capability broker can be in the form of text and can contain an organization description, processes, and prompts. The prompts can range from instructions to generate capability maps based on textual descriptions in attached files to instructions to identify goals and processes with the capability descriptions. The prompts can also contain several documents and instructions to combine them. These prompts should also contain instructions about the structure and format of the output. For example, how many levels of capabilities to include, how they should be labeled, and in what format to present the output, e.g. text or PlantUML [13].

File uploads the purpose of creating a knowledge repository that focuses on the LLMs' response. The Capability Broker should support basic file upload in formats PDF, JPEG, PNG, DOCX. There should be a limit of how many files can be used to prevent the responses of Capability Broker from becoming too unfocused. File uploads can be used for two purposes - directly in prompts and to create a knowledge base to support Retrieval Augmented Generation (RAG). For the latter, the system should store domain-specific documents to specify context and use it as a knowledge base. For example, in the military domain, the base library should already include the NAFv4 specification, AJPs, or other relevant external knowledge sources. The knowledge base should support using meta-models as a framework for creating and representing the

output diagram, such as the NAFv4 metamodel, definitions, and examples of the modelling language. These are then used by the agent as a rule set when creating the output diagram. Capability Broker should support the user in updating (such as adding or removing files) the knowledge base (library) with additional knowledge sources to assist the LLM in identifying, for example, important relationships between capabilities, processes, and activities. The contents of the knowledge base should be available for viewing.

Capability map creation focuses on the functionality for creation of capability map. Capability Broker should be able to map sub-capabilities and sub-activities across different levels and accurately represent their relationships including adding newly identified capabilities to existing capability maps. This process should enable the analyst to review and refine the suggested capabilities, including correcting, complementing, and adjusting identified capabilities, activities, relationships, and structures. The analyst should be able to approve, reject, rename, or modify individual suggested elements and their relationships.

It should provide users with preset prompts known to yield good results and allows them to save best practice prompts.

Requirements for Capability Map display.

Capability Broker should be able to display a capability map and produce a file containing the diagram image, e.g. as PDF file, which the user can save for viewing. A key function that supports diagram creation is the ability of the LLM component to generate syntactically and semantically correct PlantUML code from which a graphical diagram is generated by another component such as a PlantUML visualization tool or a modeling tool such as Astah. The interaction between these components should be achieved via an API and, in addition, the generated PlantUML code should be possible to copy to clipboard.

Since Capability Broker uses PlantUML tools for the purpose of displaying capability maps the UML class diagram notation is used, which imposes challenges to visually distinguish the components in the capability map. Therefore, Capability Broker should distinguish between classes that represent capabilities and those that represent activities, for example, by labelling them in different colors.

The interaction with Capability Broker is expected to be iterative. Hence, it should save the history of the current conversations' PlantUML code, so that it can be appended to the request if the user wants to make changes to a specific iteration of a diagram. Capability Broker should save the current conversation history of prompts and responses. Outputs should be saved as context for the agent.

We have observed that on some occasions the PlantUML code generated contains errors. Therefore, Capability Broker needs to handle situations where the agent responds with non-compiling PlantUML code. This can be communicated to the user, prompting them to try again. Alternatively, retry behind the scenes until it works.

Requirements for capability expansion.
Capability Broker should be able to match capabilities to other existing capabilities, match capabilities to other existing processes. For example, to match newly identified capabilities to already-defined capabilities based on user input, identifying which ones support each other. It should also be able to match newly identified capabilities to an already-defined process based on user input, identifying which ones support each other.

Capability Broker should be able to re-generate the output diagram, taking into account the specified feedback from the user in an iterative process. For example, if the user wants certain classes to be renamed in the diagram. It should take previous messages into account as context when generating the response.

4 The initial prototype

4.1 Example case

In this initial design cycle, the feasibility of the Capability Broker was explored through a capability mapping case in the military domain. The military domain provides a relevant context since it combines organizational complexity, substantial use of capability-oriented tactical planning, and reliance on large bodies of textual documentations, e.g., manuals and processes outlined in specific publications.

At the same time, military capability development is commonly guided by established frameworks, such as NAF, which provides modeling structures for representing capabilities and their relationships. However, descriptions of organizational activities, operational functions, and emerging needs are not always expressed in capability terms, which makes identification and structuring of capabilities challenging.

The mapping case in the military domain is also motivated by the need to assess how emerging technology changes operational requirements in relation to existing capabilities. A new capability may contribute broadly by enhancing multiple sub-capabilities, thereby strengthening the overall capability. Alternatively, it may improve only a limited part of the capability structure, e.g., a specific activity or process. Being able to identify such differences is of importance when assessing a capability's operational impact, and as a result, make informed decisions regarding prioritizing development efforts, and balancing acquisition resources. From this perspective, Capability Broker is not only designed to identify and analyze new capabilities, but also to support designers' reasoning regarding where and how a new capability affects the overall capability structure.

Hence, there is a need to quickly discover capabilities from sources that are not structured according to NAF, and to align, and where relevant match, emerging capabilities with the existing capability map in order to support analysis and assessment.

4.2 Overview of the LLM component

Regarding the LLM component of Capability Broker, the platform selected was LeChat, developed by Mistral AI. The choice of Mistral AI over other providers, such

as OpenAI, was motivated by its European origin and market position. Using a European company provides greater transparency into data storage and ensures data management is compliant with GDPR. Another motivation for choosing LeChat as the LLM platform is that it is relatively unexplored for modeling purposes compared to platforms such as ChatGPT. This section provides an overview of the LLM component and explains how it fits into the overall tool architecture.

The LLM component is implemented through what is commonly known as an AI Agent. In the field of Artificial Intelligence (AI), an agent is a computational entity that considers an environment when taking actions or making decisions [14]. The agent is commonly defined by a persona that describes its role and any additional tools it can use to perform its tasks [15]. The agent implemented in LeChat was defined by a set of definitions, instructions, and guardrails. Given the multiple definitions of what a capability is, it was necessary to establish a clear and consistent definition for the agent to follow. The relationships between a sub-capability and a capability, and between an activity and a capability, were also clearly specified to ensure the agent could understand the intended structure of the output. In terms of instructions, the workflow was specified to clarify both the input and output. Additionally, diagram rules were defined to control how LeChat is to structure the diagram and represent the capabilities and their relationships. To underline these guidelines, an example was provided for LeChat to follow as a reference. To enhance performance, guardrails were established to ensure LeChat produces accurate and well-structured outputs, maintains consistency, manages errors effectively, and continuously improves through user feedback. The definitions, instructions, and guardrails are presented below. It is important to note that their structure was suggested by an LLM to improve how it interprets and understands the information. See table 1 for details.

Table 1. Examples of prompts from a conversation for capability mapping

Definitions	
- **Capability**: The ability to achieve a desired effect under specified standards/conditions, realized through ways and means. - **Sub-capability**: A distinct, reusable ability derived from a capability's attribute. - **Activity**: A specific action or task realizing a capability.	
Instructions	**Guardrails**
## Rules for Input - Suggest clearer wording for ambiguous questions. - Example: If the input is ambiguous, re-phrase it for clarity. ## Project Workflow 1. **Input**: PDF/plain text. 2. **Output**: PlantUML class diagram (no methods) with: - Capabilities (goals + descriptions). - Notes for context. - Activities (labeled separately). - Sub-capabilities (if attributes represent distinct abilities).	## Input Handling - **Clarity**: If an input is ambiguous, ask for claification. - Example: "Could you please clarify what you mean by...?" - **Format**: Specify the preferred format for inputs. - Example: "Please provide the data in JSON format." ## Output Quality - **Accuracy**: Always verify information from reliable sources before providing an answer. - **Relevance**: Ensure that responses directly address the user's query.

## Diagram Rules 	
- Label relationships clearly (e.g., "depends on").
- Validate: Every capability must have a goal/description.
- The diagram should have multiple levels of sub-capabilities.
 	
## Examples 	
### Input Example 	
"The system must process user data and generate reports."
### Output Example 	
``plantuml
@startuml
class System {
+processUserData()
+generateReports()}
note top of System
The system must process user data and generate reports.
end note
@enduml
	- **Structure**: Use clear headings and bullet points to organize information.
	## Consistency
	- **Tone and Style**: Maintain a professional and helpful tone in all responses.
	- **Terminology**: Use consistent terminology throughout interactions.
	## Error Handling
	- **Error Messages**: Provide clear and helpful error messages.
	- Example: "I'm sorry, but I couldn't understand your request. Could you please rephrase it?"
	- **Fallback Mechanisms**: If the agent cannot resolve an issue, provide a fallback response.
	- Example: "I'm unable to assist with that request. Please contact support for further help."
	## Feedback Mechanism
	- **User Feedback**: Encourage users to provide feedback on the agent's performance.
	- Example: "How was your experience with this interaction? Please let us know how we can improve."
	- **Continuous Improvement**: Regularly review feedback and make necessary improvements.

In addition to definitions, instructions, and guardrails, external knowledge in the form of PDF documents was incorporated as a knowledge source for LeChat. This was done to reduce the chance of misinterpretations and mitigate what is commonly referred to as hallucinations in AI systems. For the purpose of this paper, the external knowledge consisted of the NAF version 4 specification. NAFv4 was selected based on its availability of documentation and relevance to the domain investigated in this paper.

The LLM components' role in the tool's overall architecture is situated in the tool's frontend, where user prompts and input files are sent to LeChat for capability identification and the construction of the capability map. The response from LeChat is then processed in the backend, where it is forwarded to a PlantUML drawing tool to generate a visual representation of the capability map, which is then presented to the user.

4.3 Prompting examples

Table 2 shows examples of prompts from a conversation for capability identification and mapping.

Table 2. Examples of prompts from a conversation for capability mapping

Prompt (Pr) /prompt chain, examples of the prompts used	**Comments on the response**
I want to find activities and capabilities specifically relating to phase 2 in the attached file. The overarching capability should be	Whilst the diagram as seen in figure 2 is not ideal, it provides a foundation for future iteration. The diagram produced is in line

"JointTargeting". It should have one sub-class that is Phase2	with the prompt given to the Agent.
Can you make every activity a sub-class to phase 2. Also make a differentiation if something is an activity or a capability.	This diagram generated provided a significant structural improvement compared to the first diagram. The contents of the Phase2 class were made into separate classes with associations to the Phase2 class. The agent also added two new capabilities for TargetDevelopment and TargetValidation.
Can you check if you find any more activities that relate to phase 2? Capabilities should be represented as a sub-class of an activity if the capability relates directly to the activity.	The agent re-organised the diagram. New relationships between capabilities and activities were created, e.g. the agent created a relationship between the activity IdentifyValidTargets and the capability TargetDevelopment.
Could you check if there are any more capabilities in the document that support the different activities?	The agent identified new capabilities such as TargetSelectionStandards and linked it to the activity of IdentifyValidTargets. In total the agent identified five new capabilities.
Could you check the document for more things that could be seen as a capability, use the following definition when checking if something could count as a capability: "A capability is the ability to achieve a desired effect under specified standards and conditions. A capability is realized through combinations of ways and means. The ability of one or more resources to deliver a specified type of effect or a specified course of action."	This rendition of the diagram was a bit of a side-step, the agent re-cassified all activities as capabilities.
No, this isn't quite right! I still want activities to be in the diagram. I wanted to check what capabilities support different activities.	The agent reverted the classification changes made in the previous rendition and added capabilities related to AdvancedTargetDevelopment. The agent added a new activity for IntelligenceStaffResponsibilities
I'm very happy with the diagram, but can you please remove the note and the execute() methods	The agent used class diagram notation and added operations to capabilities modeled as classes that needed to be removed. The agent removed as requested in the prompt.

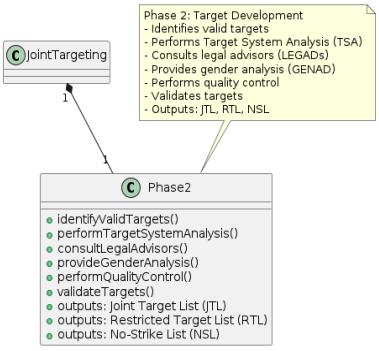


Fig. 2. The initial diagram that was produced by the agent.

The LLM generated a PlantUML code which was then visualized by a drawing tool, as shown in fig 2. The structure of this initial diagram is far from what can be expected as a capability map, however the contents of the diagram provide a good baseline. After multiple iterations and asking the agent to implement changes to the diagram the result is shown in Fig. 3.

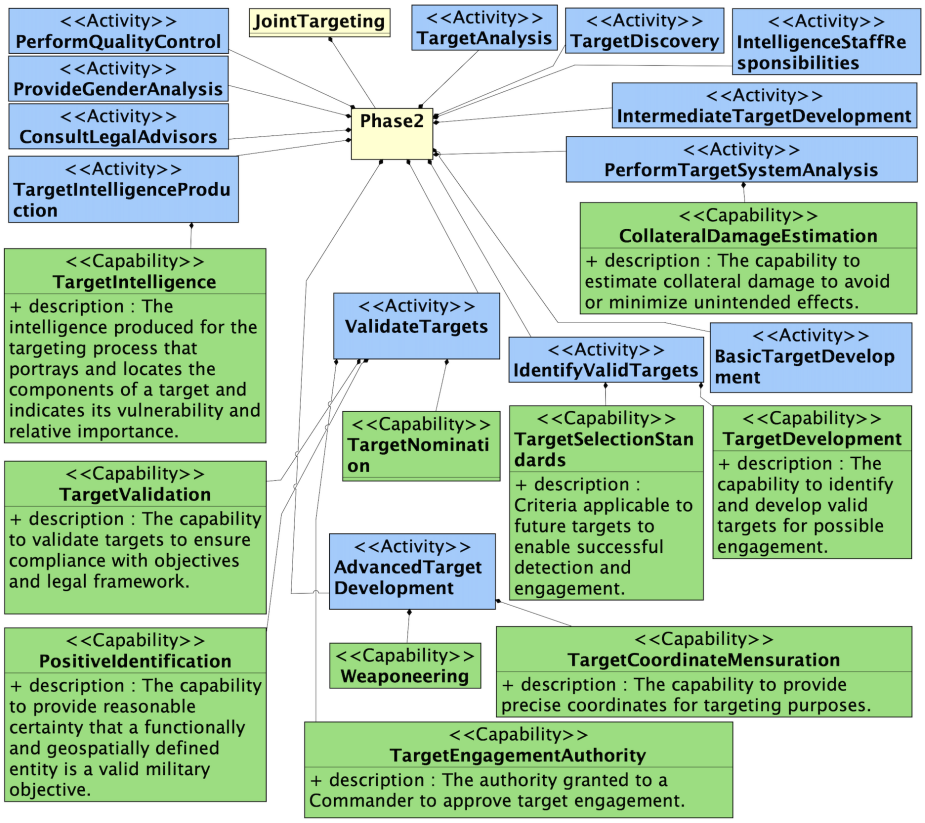


Fig. 3. The final diagram iteration.

This diagram (fig.3) shows activities of phase 2 Joint Targeting and capabilities that support the activities. According to domain expertise the capabilities are correctly identified. It was created by importing the PlantUML code into the Astah UML modeling tool and then edited to make it fit the format of this article. Without editing the diagrams that are automatically generated from the PlantUML code are very wide and would not be readable.

During the course of this design iteration, we tried several prompt chains and experimented with various input documents. It was noted in previous attempts at generating a diagram that the LLM seems to perform better when given the name of the top-level capability. This structural guidance provided a more coherent response from the LLM. The iterative approach via breaking down the problem into smaller sub-tasks had a positive effect on the output, compared to the agent trying to generate the entire diagram in one go.

5 Discussion, Conclusions, and Future Work

In this idea paper we have presented the initial requirements and feasibility analysis for Capability Broker. We have observed that capability discovery and mapping with the help of the Large Language Model (LLM) is possible to support with such a tool and that it is able to support the work of capability analysts. It is, however, not a replacement for human capability analyst and in this regard, we envision it as an assistant that analyses documents and proposes a capability map and/or capabilities that could be added to it. A particular impact of Capability Broker would be cases when new capabilities need to be discovered and aligned with the existing map.

Concerning limitations of this study two aspects need to be mentioned. First, the requirements have been elicited by means of action research and come mostly from one domain. Second, the outputs of the LLM have been compared with domain knowledge and documents but they have not been evaluated for precision and recall because a gold standard has not been established for this problem domain. Both of these limitations will be addressed in the following design cycles of Capability Broker.

Regarding future work, we plan to elaborate and experiment further with various types of input documents such as rich text descriptions of capabilities and with other LLMs. Future versions of Capability Broker need to be able support continuous development of its recommendation logic based on feedback provided by the analyst. It shall be able to use “analyst-approved” corrections and refinements to improve future suggestions/cycles. This would also need to evolve towards a round-trip integration with modeling tools, where models from a modeling tool are improved by the LLM component and then entered back into the modeling tool.

References

1. Greefhorst, D., & Proper, E. (2011). The role of enterprise architecture. In Architecture principles: the cornerstones of enterprise architecture (pp. 7-29). Springer

2. Sandkuhl, K., Stirna, J.: Capability Management in Digital Enterprises. Springer International Publishing (2018)
3. North Atlantic Treaty Organisation. (2018). NATO Architecture Version 4. AC/322-D(2018)0002-REV1, North Atlantic Treaty Organisation.
4. Andersson, K. E., Lundberg, J., & Stirna, J. (2024). Emerging technology calls for a systemic view on military capability. In PoEM Companion, vol.3645, CEUR-WS.org
5. Ulrich, W., and Rosen, M. The Business Capability Map: The 'Rosetta Stone' of Business/IT Alignment. Enterprise Architecture, Vol. 14/2 (2014)
6. Zdravkovic, J., Stirna, J., Tsai, C.H. (2025). Pondering on Capability Brokering with LLM. In: Pufahl et al.(eds) Intelligent Information Systems. CAiSE 2025. LNBIP 557, Springer
7. Storey V.C., Pastor O., Guizzardi G., Liddle S.W., Maaß W., Parsons J., Ralyté J., Santos M.Y., Large language models for conceptual modeling: Assessment and application potential, Data & Knowledge Engineering, vol. 160, 2025, ISSN 0169-023X
8. Hevner, A., March, S.T., Park, J., Ram, S. (2004) Design Science in Information Systems Research. Management Information Systems Quarterly, 28(1), 75-105
9. Johannesson P. & Perjons E., An Introduction to Design Science, Springer
10. Bērziša, S. et al. Capability Driven Development: An Approach to Designing Digital Enterprises. Business Inf. Syst. Engineering, 57, (2015)
11. Van Riel, J., Poels, G., Koutsopoulos, G., Calhau, R. F., Bider, I., Perjons, E., Wautelet, Y., & Tsilionis, K. (2025). Advancing the Domain of Strategy Planning and Implementation through Enterprise Architecture: A Research Agenda for Capability-Based Management. Communications of the Association for Information Systems, 56, 155-166
12. Capability Broker, https://github.com/Qp0p/Capability-Broker-diagrams-and-requirements
13. Drawing UML with PlantUML, PlantUML Language Reference Guide (Version 1.2025.0), https://plantuml.com/guide
14. Xi, Z. et al. (2025). The rise and potential of large language model based agents: A survey. Science China Information Sciences, 68(2)
15. Masterman, T., Besen, S., Sawtell, M., & Chao, A. (2024). The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. arXiv:2404.11584.